

Team Coordination — Project 11_02

Near Real-Time Financial Forecasting using Amazon SageMaker

Arthur, Dalibor, Ilyas, Francesco, Ralph

Task split

Member	Steps	Services
Ilyas	1–2	EventBridge, Lambda (data fetcher)
Dalibor	3–4	SQS, Glue, S3
Ralph	5–6	Comprehend, SageMaker
Francesco	7–8	DynamoDB, QuickSight
Arthur	9–10	SNS, Step Functions, CloudWatch

Flow of dependencies

Ilyas → Dalibor → Ralph → Francesco

↓
Arthur (wires everything together + alerts)

- Ilyas is the starting point. Until his Lambda pushes data, the rest of us have no real input.
 - Dalibor, Ralph and Francesco can start with fake data and plug in the real one later.
 - Arthur ties the whole thing together at the end, but can already prepare SNS and CloudWatch on his side in parallel.
-

Things we need to agree on before coding

1. **AWS region** (same for all of us)
2. **Naming prefix** for everything we create (e.g. proj1102-)

3. **Tickers** we track (AAPL, TSLA, BTC, ETH...)
4. **Timestamp format:** ISO-8601 UTC
5. **S3 bucket name** (Dalibor owns it, we all read/write to agreed paths)
6. **Message format** between Ilyas and Dalibor (price + news JSON)
7. **Dataset format** between Dalibor and Ralph (for SageMaker)
8. **DynamoDB format** between Ralph and Francesco
9. **How often** SageMaker retrains (hourly or daily, depending on cost)

Once we agree on these, let's not change them without telling the group.

Handoffs — who gives what to who, with a simple example

Ilyas → Dalibor

Ilyas gives Dalibor: - The SQS queue names and their ARNs - The shape of the JSON messages (price and news)

Example: At 10:00, Ilyas's Lambda wakes up, calls Yahoo Finance and gets AAPL at 185.42. It packages this into a small JSON message and drops it into the prices-queue. Same thing for a news headline going into news-queue. Dalibor's side then picks it up from there. So Dalibor only needs to know the queue name and what the JSON looks like.

Dalibor → Ralph

Dalibor gives Ralph: - The S3 bucket and the path where the clean data lands - The file format (Parquet or CSV) and the column names

Example: Dalibor's Glue job takes all the raw messages that arrived in SQS during the last hour, cleans them, joins prices with sentiment, and writes a clean file to s3://proj1102-data/forecast-ready/AAPL.parquet. Ralph can then point SageMaker at that exact file to retrain or generate predictions. If Dalibor changes the path or renames a column, Ralph's job will break, so they need to agree on it once and keep it stable.

Ralph → Francesco

Ralph gives Francesco: - The S3 path where forecast results are stored - The DynamoDB table name and its schema (ticker, timestamp, p10/p50/p90, is_anomaly)

Example: Ralph's SageMaker pipeline finishes a prediction round and writes the output to S3, and also writes the latest predicted range (with anomaly flag) to DynamoDB. Francesco then reads from S3 to build the QuickSight dashboard (history + charts), and uses DynamoDB for anything needing "the latest value right now". If Ralph changes the column names or the table structure, Francesco's dashboard or DynamoDB reads will stop working.

Francesco → Arthur

Francesco gives Arthur: - The DynamoDB anomaly flag format (so the alerting Lambda knows how to read it) - The QuickSight dashboard URL

Example: Francesco writes `is_anomaly: true` in DynamoDB when the real price is outside the predicted range. Arthur's Lambda reads this flag, and if it's true, sends an alert through SNS (email or SMS). So Arthur only needs to know: where to look and what to look for.

Everyone → Arthur

All of us give Arthur: - The ARN of every Lambda, Glue job and SageMaker resource we create (he needs them for Step Functions) - The name of our log groups (he needs them for CloudWatch)

Example: Arthur builds one Step Functions state machine that does: Ilyas's Lambda → Dalibor's Glue → Ralph's SageMaker → Francesco's DynamoDB → Arthur's alert. To wire that together, he literally needs the ARNs of each piece. Same for CloudWatch: he can only monitor our stuff if he knows where our logs live.

Order of work

Week 1 — Everyone in parallel - Agree on the points above - Each person builds their own part using fake/mock data

Week 2 — Pairwise integration - Ilyas ↔ Dalibor: real messages flowing through SQS - Dalibor ↔ Ralph: clean data in S3, SageMaker running on real data - Ralph ↔ Francesco: predictions and anomalies in DynamoDB, dashboard showing them - Francesco ↔ Arthur: anomaly → SNS alert

Week 3 — Arthur wires everything - Step Functions orchestrates the full pipeline - CloudWatch dashboard + alarms

Week 4 — Polish, run it for 24h, final report + demo

Shared tracker

Let's keep a Google Doc with this table and fill it in as we create resources. That way no one has to dig through the AWS console to find each other's stuff.

Resource	Name	Owner	ARN / URL
Lambda fetch	proj1102-fetch-data	Ilyas	
SQS prices	proj1102-prices-queue	Dalibor	
SQS news	proj1102-news-queue	Dalibor	
Glue job	proj1102-etl	Dalibor	
S3 bucket	proj1102-data	Dalibor	
Sentiment Lambda	proj1102-sentiment	Ralph	
SageMaker	proj1102-predictor	Ralph	
DynamoDB	proj1102-forecasts	Francesco	
QuickSight	proj1102-dashboard	Francesco	
SNS topic	proj1102-alerts	Arthur	
Step Function	proj1102-pipeline	Arthur	

Communication

- One group chat for daily updates

- One shared doc for ARNs and names (link above)
- 2 short syncs per week (15 min) to unblock each other
- Any change in schema or naming → tell the group right away